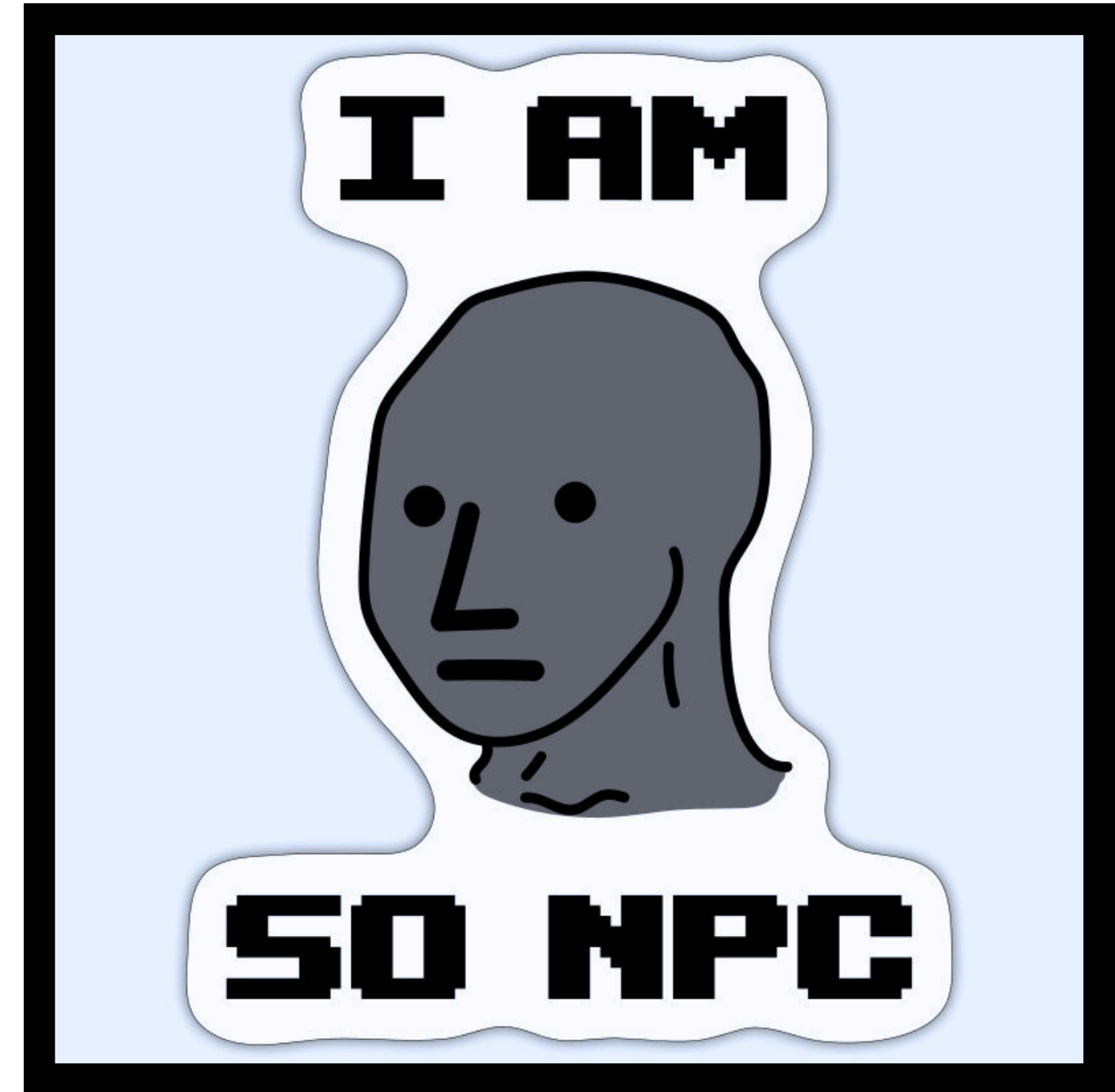
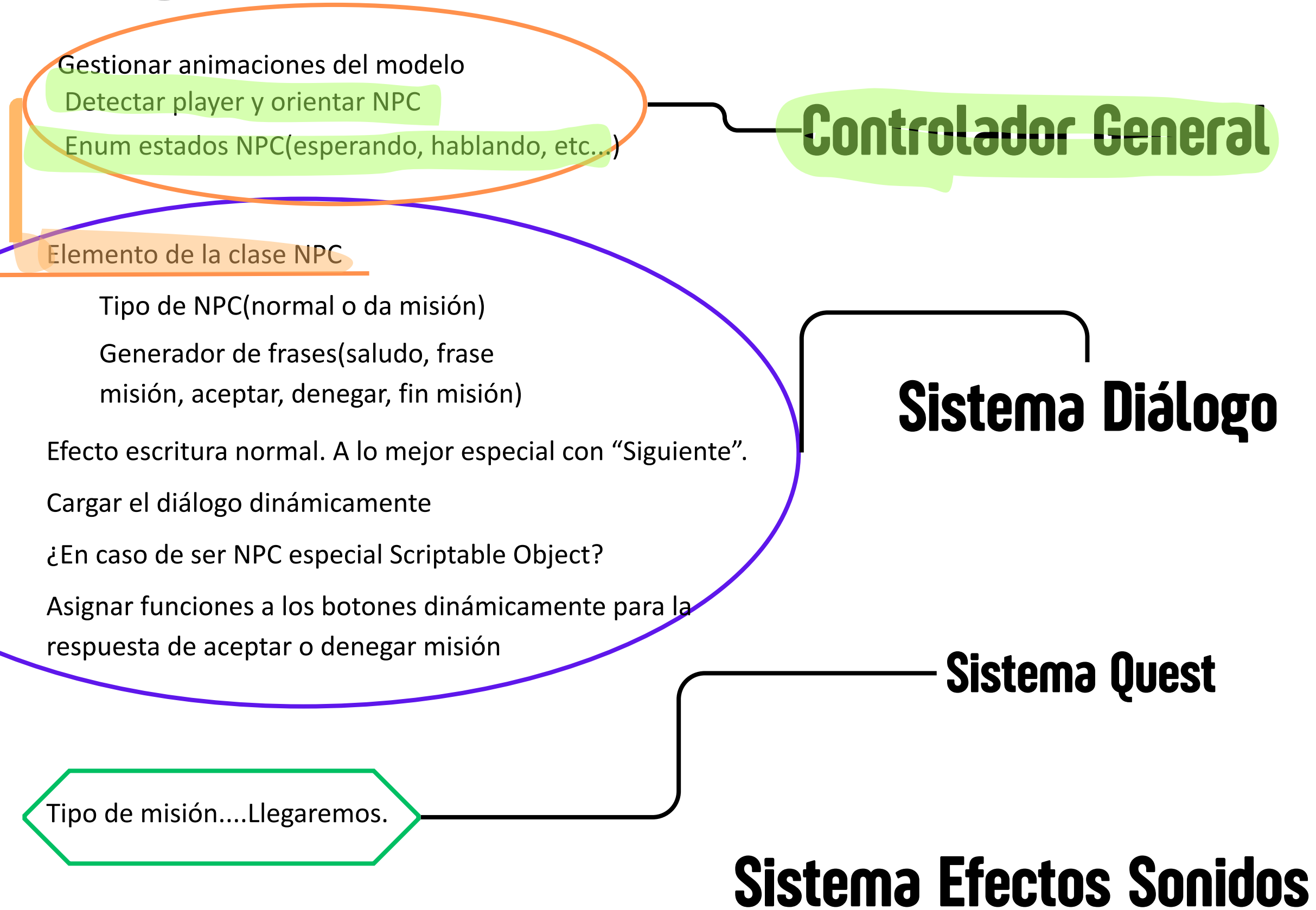
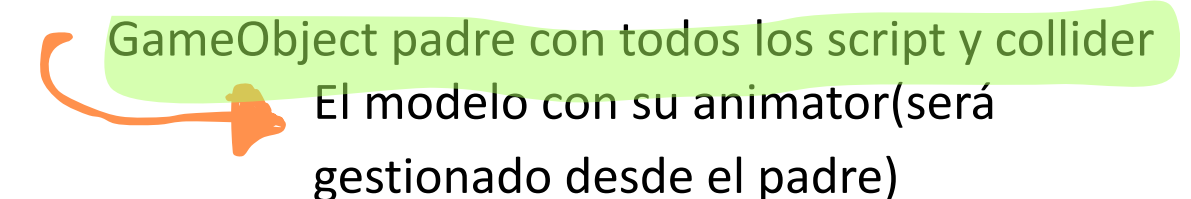


¿Qué tiene un NPC?

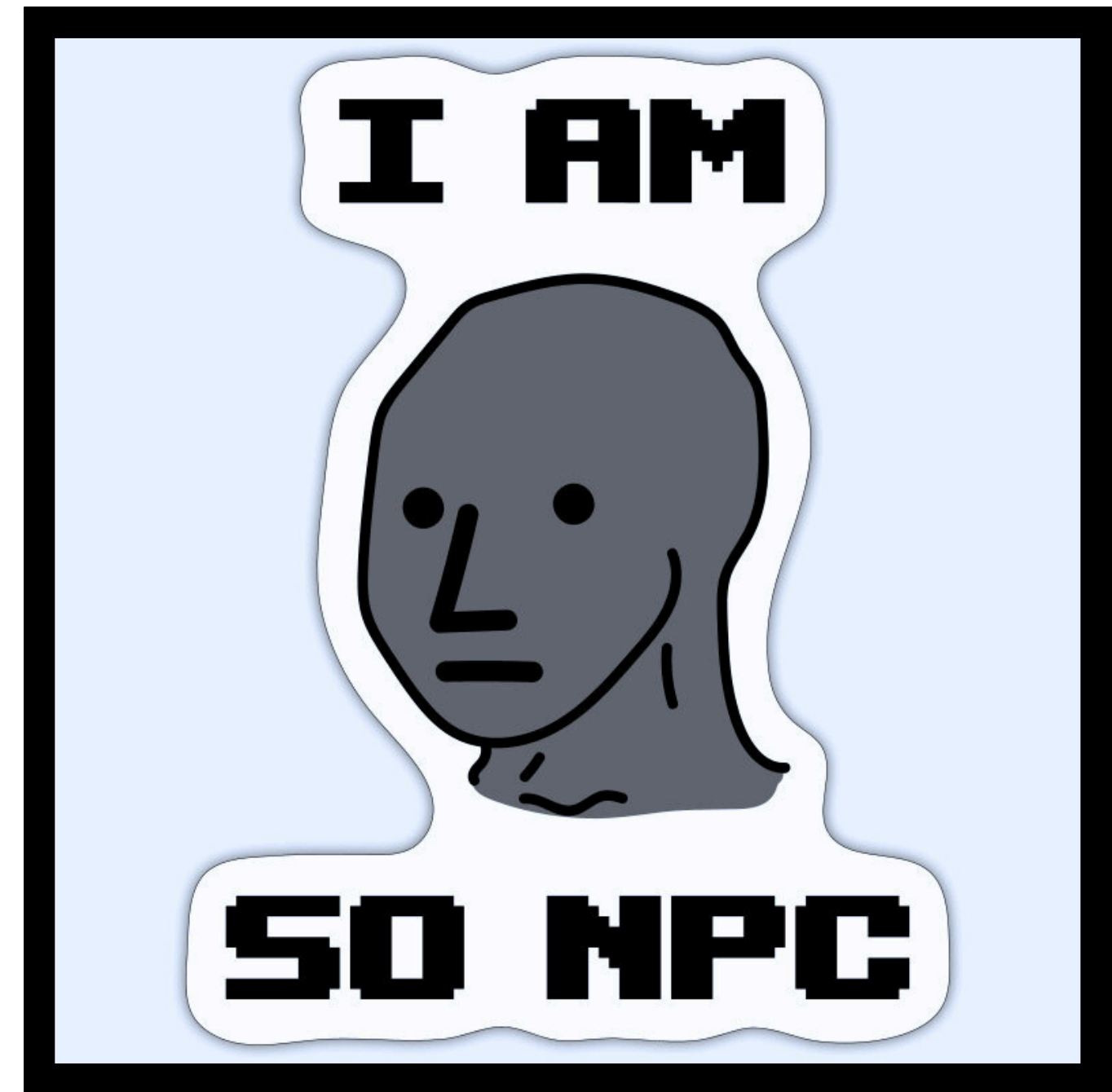


Estructura:



Nota: Mismo nombre para todos los NPC en los nodos del animator

Sistema de conversación



Tendremos que tener en cuenta:

Efecto Escritura

Controlar si está hablando el NPC

Guardar la info del elemento de la clase NPC

Asignar por código las funciones del NPC a los botones

¿Qué tiene una misión?!!!

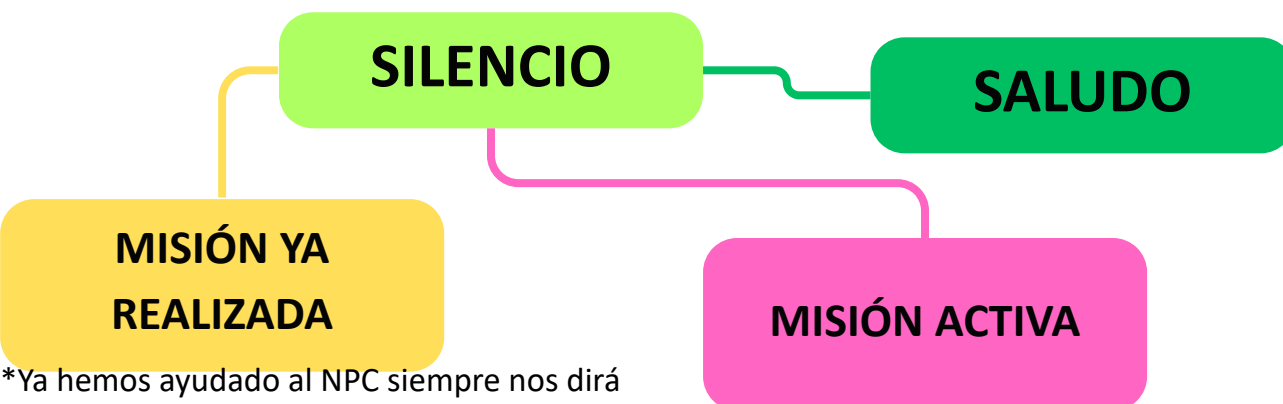


Mismo atributos que el scriptable pero además:

- Otros(a la espera de desarrollo)

Este objeto irá en el script Sistema Quest del NPC que comprobará si el NPC tiene una misión que dar y el estado de la misma.

Además, tendremos que reconfigurar la conversación del NPC teniendo en cuenta varios nuevos escenarios a la hora de interactuar con un npc desde el punto “Silencio” como son:



Atributos que cargaremos manualmente:

- Id misión
- Primera Frase / Frase a mostrar en UI
- ¿Recompensa?
- Tipo Misión(Enum)
- Objeto especial(si instanciaremos un objeto especial para lograr la misión(**concepto en obras**))
- TODOS los campos necesarios si es de recolecta:
 - Prefab
 - Puntos del mapa
- TODOS los campos necesarios si es de puzzle:
 - Prefab a instanciar en UI
- TODOS los campos necesarios si es de combate:
 - ***En Obras***
- Atributo “Misión Especial”(scriptable) para una posible misión especial autogestionable.



NOTA:

La complejidad es absoluta y solo podemos llegar hasta donde planteemos jugando una y otra vez con ¿Qué tiene una misión?

La forma correcta de desarrollar todo esto es con clases, herencias y objeto abstractos(entre ellos los scriptables) pero ni los hemos visto ni caemos en burbujas mediáticas de la ultra derecha

*Ya hemos ayudado al NPC siempre nos dirá “Gracias por haberme ayudado”

SISTEMA DE CONBATE BIEN MAMÓN

- Datos(Stats) preconfigurados de un personaje

SCRIPTABLE FICHA COMBATE

CLASE FICHA COMBATE

SCRIPTABLE ATAQUE

- Enum tipo ataque(ataque, aumento stat, invocación...)

- Stats e información relevante para el personaje
- Un scriptable para cargar la info anterior.
- Una lista de cuatro scriptables de tipo ataque
- Posible ataques
- IComparable que compare por el stat "Velocidad"

```
public ScriptableAtaque[] ataques;

public void AumentarDefensa(int cantidad)
{
    defensa += cantidad;
    Debug.Log($"Defensa aumentada en {cantidad}. Defensa actual: {defensa}");
}

public void UsarAtaque(int indiceAtaque)
{
    ataques[indiceAtaque].Ejecutar(this);
}
```

```
public void Ejecutar(Personaje objetivo)
{
    if (aumentaDefensa)
    {
        objetivo.AumentarDefensa(
            cantidadDefensaExtra);
    }
}
```

PERFIL COMBATE

CONTROLADOR CÁMARAS

- Controla combat camera

CONTROLADOR UI COMBATE

- Controla el UI del combate
- Cortinilla

NUEVOS ENUMS

- Tipo Ataque
- Aliado o Enemigo
- Fase Combate(Sin asignar, Comenzando, Combatiendo, Resuemn y Fin)
- Modificar enum Estado General-añadir Combatiendo
- Enum tipoTurno para controlar lo del Player o Enemigo

CONTROLADOR ABSOLUTO COMBATE

- Lista de objeto Perfil Combate con toda la info de los combatiente
- Ordena la lista en base a la velocidad
- Enum tipo Fase Combate(Sin comenzar, comenzando, etc...)
- Enum tipo tipoTurno(controla fase Player o Enemigo)
- Gestiona el movimiento y el ataque por turno
- Variable bool ha atacado para controlar las acciones del personaje(el total de movimiento los sabemos por los puntos de movimiento restantes)
- Comunicación con el sistema de cámaras y UI combate entre otros
- Variable objetivo al que se le va a atacar en este turno(Puede ir aquí también 2/2)

SCRIPTABLE TIPO MISIÓN "COMBATE"

- Nuevo tipo de misión
- Tendrá un vector3 para la cámara
- ¿Frase victoria o derrota?

